



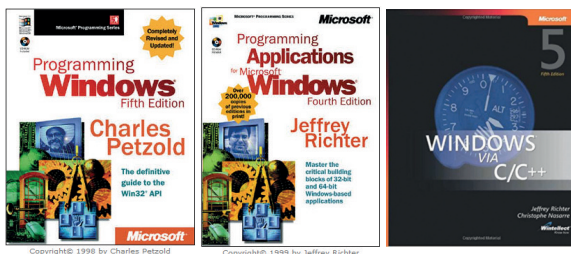
Christophe PICHAUD
Consultant sur les technologies Microsoft
christophepichaud@hotmail.com |
www.windowscpp.com



Windows - Le Multithreading en C/C++

Le mois dernier, nous avons vu comment faire des applications multithreads en C++ sous Linux avec la chaîne de compilation GNU de GCC. Ce mois-ci on fait du spécifique Windows mais remarquez que tout ce qui a été écrit compile aussi sur Windows car on avait utilisé la STL du C++ et qu'elle est portable au niveau source. Le C++ est ISO et multiplateforme.

Sous Windows dans les années 199x, là où Windows était en plein essor, la programmation système était décrite dans le livre de Charles Petzold, « Programming Windows ». Cet ouvrage que de nombreux développeurs ont dévoré décrit l'utilisation des API Windows et l'un des chapitres dans la section « Advanced Topics » est le multithreading. L'autre auteur à succès était Jeffrey Richter et son ouvrage uniquement sur le Win32 système. Pour ceux qui sont intéressés, le livre de Jeffrey Richter est devenu Windows via C/C++...



Un petit retour sur l'avant Win32. Avant Windows 95 et Windows NT, le seul moyen de faire des opérations en arrière-plan était d'utiliser un objet Timer. Le nombre d'objets était limité mais cela permettait déjà de réaliser des choses de bonne facture. Avec Win32, le système s'est doté de la notion de threads. Le fondateur et Architecte de Windows NT, Dave Cutler, a créé le système à l'image des OS de Digital de l'époque avec la notion de sécurité au niveau des objets Kernel. Je m'explique : un thread est un objet Kernel. Ce n'est pas comme une fenêtre GDI. C'est identifié par un type HANDLE et il faut absolument le gérer proprement sinon on dégrade le système. Un HANDLE est obtenu avec la primitive CreateThread et il doit être libéré via CloseHandle. Cette primitive est définie dans Windows.h :

```
HANDLE WINAPI CreateThread(
    _In_opt_ LPSECURITY_ATTRIBUTES lpThreadAttributes,
    _In_     SIZE_T dwStackSize,
    _In_     LPTHREAD_START_ROUTINE lpStartAddress,
    _In_opt_ LPVOID lpParameter,
    _In_     DWORD dwCreationFlags,
    _Out_opt_ LPDWORD lpThreadId
);
```

Un thread est une opération (une fonction alias une routine C) lancée en arrière-plan et ordonnancée par le système d'exploitation. A première vue, vous allez me dire, CreateThread ce n'est pas évident à utiliser... En fait un bon sample va faire l'affaire pour bien

comprendre. Le premier paramètre est une structure de descripteur de sécurité et peut être NULL. Le second est la taille de la pile. S'il est positionné à 0, on aura la taille par défaut. Le troisième est la fonction du thread. Le quatrième est le pointeur vers les paramètres à passer au thread. Le cinquième est un ensemble de flags pour dire si le thread est suspendu ou démarré immédiatement : 0 ou CREATE_SUSPENDED. Le dernier paramètre retourne l'Id du thread. Enfin, la fonction retourne le HANDLE du thread si la fonction a réussi sinon c'est NULL. Un sample ?

```
#include <Windows.h>

DWORD WINAPI MyThreadProc(LPVOID lpParam)
{
    for (int i = 0; i < 10000; i++)
    {
        printf(" ");
    }

    printf("\n");
    return 0;
}

int main()
{
    DWORD dwThreadId = 0;
    HANDLE hThread = CreateThread(NULL, 0, &MyThreadProc, NULL, 0, &dwThreadId);
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
    return 0;
}
```

Comme vous pouvez le constater, il y a un peu de glue pour attendre la fin du thread via la primitive WaitForSingleObject... Ce type de code fonctionne depuis Windows NT et Win95 et fonctionnera toujours. Les API système ne sont jamais marquées *depreca-*ted !

Une variante en Runtime C

Il existe une variante à CreateThread dans la runtime du C, la CRT. C'est via les fonctions _beginthread et _beginthreadex. Voici la variante avec _beginthread :

```
#include <stdio.h>
```

```
#include <process.h>
#include <Windows.h>

void MyThreadProc(void* pArguments)
{
    for (int i = 0; i < 10000; i++)
    {
        printf(".");
    }

    printf("\n");
}

int main()
{
    unsigned int threadID = 0;
    HANDLE hThread3 = (HANDLE)_beginthread(&MyThreadProc, 0, NULL);
    WaitForSingleObject(hThread3, INFINITE);
    CloseHandle(hThread3);

    return 0;
}
```

La variante avec `_beginthreadex` n'a que très peu d'intérêt car elle est calquée exactement sur `CreateThread`:

```
unsigned int threadID = 0;
HANDLE hThread2 = (HANDLE)_beginthreadex(NULL, 0, &MyThreadProc, NULL, 0, &threadID);
WaitForSingleObject(hThread2, INFINITE);
CloseHandle(hThread2);
```

Si une routine veut se mettre en attente ou donner la main aux autres threads, il suffit d'appeler la fonction `Sleep(xxx ms)` ;

```
DWORD WINAPI MyThreadProc(LPVOID lpParam)
{
    for (int i = 0; i < 100; i++)
    {
        printf(".");
        Sleep(50);
    }

    printf("\n");
    return 0;
}
```

Cela permet de contrôler la vitesse d'exécution du thread.

La terminaison d'un thread

Un thread se termine de plusieurs façons :

- La fonction se termine avec un code retour ;
- Le thread se tue lui-même via `ExitThread` ;
- Un thread dans le même processus ou un autre appelle `TerminateThread` ;
- Le processus qui contient le thread se termine.

Suspendre ou résumer un thread

La routine `ResumeThread(HANDLE hThread)` permet de résumer un thread.

La routine `SuspendThread(HANDLE hThread)` permet de suspendre un thread.

La priorité d'un thread

Il est possible d'augmenter la priorité d'un thread via `SetThreadPriority`. Attention, il faut bien expérimenter cette fonction avant de l'utiliser en production !

Thread Priority	Symbolic Constant
Time-critical	THREAD_PRIORITY_TIME_CRITICAL
Highest	THREAD_PRIORITY_HIGHEST
Above normal	THREAD_PRIORITY_ABOVE_NORMAL
Normal	THREAD_PRIORITY_NORMAL
Below normal	THREAD_PRIORITY_BELOW_NORMAL
Lowest	THREAD_PRIORITY_LOWEST
Idle	THREAD_PRIORITY_IDLE

Les routines Interlocked

Il existe des routines pour faire des opérations ATOMIC. Exemple : pour incrémenter, décrémenter une variable. Il s'agit de la série `Interlocked_FUNCTION_NAME` comme `InterlockedIncrement`, `InterlockedDecrement`, `InterlockedExchange`, etc.

La synchronisation des threads en mode User

Pour synchroniser des threads, on peut passer par des booléens par exemple mais à condition de les protéger. Les sections critiques servent à cela via les routines :

- `InitializeCriticalSection`
- `EnterCriticalSection`
- `LeaveCriticalSection`

```
long _x = 0;
long _y = 0;
CRITICAL_SECTION _cs1;

DWORD WINAPI MyThreadP1(LPVOID lpParam)
{
    InitializeCriticalSection(&_cs1);

    for (int i = 0; i < 10000; i++)
    {
        EnterCriticalSection(&_cs1);
        _x++;
        _y++;
        printf("%d/%d.", _x, _y);
        LeaveCriticalSection(&_cs1);
    }

    printf("\n");
    return 0;
}
```

```

}

int main()
{
    DWORD dwThreadId1 = 0;
    HANDLE hThread1 = CreateThread(NULL, 0, &MyThreadP1, NULL, 0, &dwThreadId1);
    DWORD dwThreadId2 = 0;
    HANDLE hThread2 = CreateThread(NULL, 0, &MyThreadP1, NULL, 0, &dwThreadId2);
    DWORD dwThreadId3 = 0;
    HANDLE hThread3 = CreateThread(NULL, 0, &MyThreadP1, NULL, 0, &dwThreadId3);
    HANDLE hTab[3] = { hThread1, hThread2, hThread3 };
    WaitForMultipleObjects(3, hTab, TRUE, INFINITE);
    CloseHandle(hThread1);
    CloseHandle(hThread2);
    CloseHandle(hThread3);

    return 0;
}

```

Vous remarquerez que je protège les variables en écriture mais aussi en lecture !

La synchronisation des threads en mode Kernel

Il existe des mécanismes plus puissants que les sections critiques et ce sont des objets Kernel. Le plus connu étant le mutex. Ces objets sont signalables ; voici le tableau de comparaison entre un mutex et une section critique : **1**

Le Concurrency Runtime alias ConCRT

En 2011, Microsoft sort une librairie C++ pour exploiter à fond les threads des processeurs : le Concurrency Runtime : **2**

Parallel Patterns Library

PPL fournit des classes collections (containers) et des algorithmes pour réaliser des opérations parallèles et des opérations distribuées sur des ensembles de données ou des collections. PPL permet aussi le parallélisme des tâches en fournissant des objets Task réparties sur les ressources matérielles.

Asynchronous Agents Library

Agents Library permet de faire communiquer différents éléments d'un programme au travers d'interfaces et met en œuvre un pipeline évolué.

Task Scheduler

Le Task Scheduler gère et coordonne les tâches à l'exécution. Le Task Scheduler est coopératif.

1

Characteristic	Mutex	Critical Section
Performance	Slow	Fast
Can be used across process boundaries	Yes	No
Declaration	<code>HANDLE hmtx;</code>	<code>CRITICAL_SECTION cs;</code>
Initialization	<code>hmtx = CreateMutex (NULL, FALSE, NULL);</code>	<code>InitializeCriticalSection(&cs);</code>
Cleanup	<code>CloseHandle (hmtx);</code>	<code>DeleteCriticalSection(&cs);</code>
Infinite wait	<code>WaitForSingleObject (hmtx, INFINITE);</code>	<code>EnterCriticalSection(&cs);</code>
0 wait	<code>WaitForSingleObject (hmtx, 0);</code>	<code>TryEnterCriticalSection(&cs);</code>
Arbitrary wait	<code>WaitForSingleObject (hmtx, dwMilliseconds);</code>	Not possible
Release	<code>ReleaseMutex (hmtx);</code>	<code>LeaveCriticalSection(&cs);</code>
Can be waited on with other kernel objects	Yes (use <code>WaitForMultipleObjects</code> or similar function)	No

Resource Manager

Le Resource Manager a pour rôle de gérer les ressources mémoire et processeur.

Les fichiers d'entêtes suivants sont nécessaires pour tirer parti des différentes parties de CR :

Composant	Fichier d'entêtes
Parallel Patterns Library (PPL)	ppl.hpltasks.h concurrent_queue.h concurrent_vector.h
Asynchronous Agents Library	agents.h
Task Scheduler	concr.h
Resource Manager	concrtrm.h

L'ensemble des classes et templates qui composent ConCRT sont incorporés dans le namespace Concurrency.

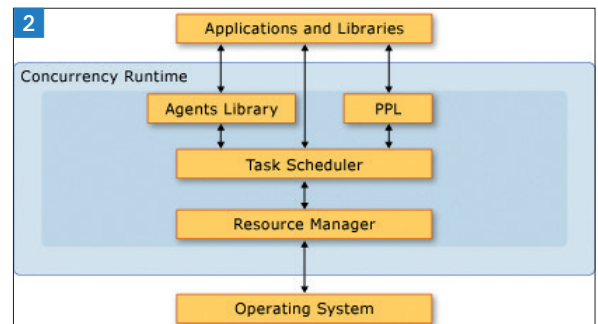
Le blocage coopératif

Avec le blocage coopératif, une tâche peut suspendre son exécution et redonner la main au Task Scheduler pour réutiliser des ressources et donc optimiser l'usage de ces ressources pour exécuter d'autres tâches. Contrairement aux primitives de bas niveau du système d'exploitation (Windows), les ressources processeur sont optimisées. Le blocage coopératif permet d'améliorer les performances des applications parallèles en faisant une utilisation pleine des ressources CPU. Les principaux objets de synchronisation sont de type verrou simple (classe `critical_section`), verrou de type SWMR alias Single Writer Multiple Readers (classe `reader_writer_lock`), event manuel (classe `event`).

Liste des opérations de blocage coopératif :

Opérations de blocage coopératif	Description
<code>task_group::wait</code>	attend la fin d'une tâche ou d'un groupe de tâches
<code>critical_section::lock</code>	acquisition d'un verrou de type section critique
<code>critical_section::scope_lock</code>	version exception safe
<code>reader_writer_lock::lock</code>	acquisition d'un verrou de type reader/writer en écriture
<code>reader_writer_lock::scope_lock</code>	version exception safe
<code>reader_writer_lock::lock_read</code>	acquisition d'un verrou de type reader/writer en lecture
<code>reader_writer_lock::scope_lock_read</code>	version exception safe
<code>event::wait</code>	équivalent à un manuel reset event
<code>agent::wait</code>	attend la fin de l'agent
<code>agent::wait_for_all, wait_for_one</code>	attend la fin de un ou plusieurs agents
<code>Concurrency::wait</code>	blocage pendant un intervalle
<code>Context::Block</code>	interne à CR
<code>Context::Yield</code>	donne la main à un autre contexte d'exécution
<code>parallel_for, parallel_for_each, parallel_invoke</code>	fonctions pour les algorithmes parallèles
<code>send, asend</code>	fonctions de transmissions de données par messages
<code>receive</code>	Fonction de réception de message

2



Les tâches et les threads

Lorsqu'une tâche doit s'exécuter, le Task Scheduler appelle la routine de cette tâche dans un thread de son choix. La tâche ne change pas de thread. C'est une garantie pour le code ayant une affinité avec un thread ou un usage par exemple du TLS (Thread Local Storage). La création d'une tâche est une opération moins coûteuse que la création explicite d'un thread. Il est possible pour une application de créer des centaines ou des milliers de tâches tout en tournant efficacement. Ceci n'est pas possible avec des threads explicites. La conception d'une application en tâches est moins compliquée que la conception d'une application à base de threads.

Création des tâches

La création des tâches peut se faire avec des fonctions lambdas ou de simples pointeurs de fonctions. La classe `task_group` peut être utilisée de différentes manières. Pour une déclaration détaillée, la classe `task_group` utilise la méthode `run()` surchargée qui prend en paramètre un `task_handle` dans lequel on spécifie explicitement le nom de la fonction à utiliser. La syntaxe moderne du style C++ autorise l'utilisation du mot clé `auto` pour masquer le type retourné par la fonction `make_task`. La nouvelle norme C++ introduit des classes templates pour la gestion des pointeurs de fonctions et il est possible d'exploiter toutes ces formes d'écriture dans une même fonction. Cependant, les fonctions à utiliser doivent avoir une signature de type `void function(void)`.

```
void DoSimpleTask(void)
{
    Logger::LogDebug("Enter DoSimpleTask...");
    wait(1000);
    Logger::LogDebug("Leave DoSimpleTask...");
}

void Call_Task_Group_Run()
{
    // Déclaration du Task Group
    task_group g;
    // Avec une déclaration explicite de task_handle et de prototype
    task_handle<void (*) (void)> t1 = make_task(&DoSimpleTask);
    // Avec auto
    auto t2 = make_task(&DoSimpleTask);
    // Avec std::tr1::function
    std::function<void (void)> f3 = &DoSimpleTask;
    auto t3 = make_task(f3);
    // Création des tâches
    g.run(t1);
    g.run(t2);
    g.run(t3);
    g.run(&DoSimpleTask);
    g.wait();
}
```

Le fichier d'entêtes `pppltasks.h` introduit de nouveaux templates et des nouvelles classes pour gérer les tâches. Ainsi, on y trouve le template de classe `task<>` qui prend en paramètre dans son constructeur un objet de type `std::function`. La fonction passée à la classe

`task` peut aussi retourner un type donné et ainsi ressembler à un thread classique qui retourne, généralement 0 en cas de succès :

```
int DoSimpleTask_Random()
{
    Logger::LogDebug("Enter DoSimpleTask_Random...");
    wait(1000);
    // Génération d'un nombre aléatoire de 1..1000
    unsigned int r = 0;
    srand(1); rand_s(&r);
    r = (unsigned int) ((double)r / ((double) UINT_MAX + 1) * 1000.0) + 1;
    Logger::LogDebug("Leave DoSimpleTask_Random...");
    return r;
}

void Call_Task_Group_Run_Extras()
{
    std::function<int (void)> f1 = &DoSimpleTask_Random;
    task<int> t1(f1);
    t1.wait();
    int ret = t1.get();
    printf("Tasks t1 returns %d\n", ret);

    std::function<void (void)> f2 = &DoSimpleTask;
    task<void> t2(f2);
    t2.wait();
}
```

Le `ConcRT` sample pack introduit des opérateurs qui rendent la syntaxe plus élégante. Pour créer deux tâches et attendre leur terminaison, il suffit d'utiliser l'opérateur `&&`.

```
void Call_Task_Group_Run_Extras_WithOperator()
{
    std::function<int (void)> f1 = &DoSimpleTask_Random;
    task<int> t1(f1);
    std::function<int (void)> f2 = &DoSimpleTask_Random;
    task<int> t2(f2);
    (t1 && t2).wait();
    printf("Tasks t1 returns %d\n", t1.get());
    printf("Tasks t2 returns %d\n", t2.get());
}
```

Pour enchaîner la création d'une tâche une fois que les deux premières tâches sont terminées, il suffit d'utiliser la méthode `then()` :

```
void Call_Task_Group_Run_Extras_WithOperator2()
{
    std::function<void (void)> f1 = &DoSimpleTask;
    task<void> t1(f1);
    std::function<void (void)> f2 = &DoSimpleTask;
    task<void> t2(f2);
    std::tr1::function<void (void)> f3 = &DoSimpleTask;
    auto t3 = (t1 && t2).then(f3);
    t3.wait();
}
```

La gestion du Task Scheduler

Il est possible de choisir la manière avec laquelle le ConcrRT va gérer les tâches au travers l'utilisation de threads. La classe `SchedulerPolicy` permet de spécifier le nombre minimum et maximum de ressources CPU (processeurs virtuels) via les propriétés `MinConcurrency` et `MaxConcurrency`. La propriété `SchedulerKing` permet de choisir le type de threads : Win32 ou UMS. UMS, ou User-mode scheduling, est un mécanisme léger de gestion de threads et disponible dans les versions 64 bits de Windows 7 et Windows Server 2008 R2.

```
SchedulerPolicy policy(2,  
    MinConcurrency, 1,  
    MaxConcurrency, 2);  
CurrentScheduler::Create(policy);
```

Il est aussi possible de contrôler finement la manière avec laquelle les tâches sont réparties en créant un groupe de scheduler via les classes `Scheduler` et `ScheduleGroup`. La méthode `ScheduleTask()` permet aussi de créer des tâches légères (light-weight task). La librairie Agents fait une utilisation intensive de cette méthode. ConcrRT ne gère pas et ne capture pas les exceptions générées par les tâches légères. De plus, la terminaison d'une tâche légère n'est pas signalée par le runtime ConcrRT.

Conclusion

La gestion des threads en Win32 peut sembler complexe mais les API sont relativement simples à utiliser. Le problème ne vient pas des primitives système mais de la complexité des applications multithreads, du partage de données, et des bugs pouvant en résulter en cas de non protection. Et c'est dans ce cas que le Concurrency Runtime prend toute sa place car plus simple, plus pratique. A vous de jouer ! •

reste cadre de 185 mm de large x 162 mm de haut